第二章 函数、类与数据

Lianghai Xiao

https://github.com/styluck/mlb

创建一个简单函数

- def my_function(a, b, c):
- print(a, b, c)

- def my_function(name, age, city):
- print(f"{name} is {age} years old and lives in {city}.")

利用生成器定义函数

- def my_generator():
- yield 1
- yield 2
- yield 3
- gen = my_generator()

- for value in gen:
- print(value)

*arg命令

- def my_function(*args):
- for arg in args:
- print(arg)
- # 可以将任意数量的参数传递给 *args，这里被打包成一个元组
- 。my_function(1, 2, 3)
- my_function("a", "b", "c")

反向打包

- def my_function(a, b, c):
- print(a, b, c)

- values = (1, 2, 3)
- my_function(*values) # 解包元组

反向打包

- def my_function(name, age, city):
- print(f"{name} is {age} years old and lives in {city}.")

- info = {"name": "Alice", "age": 25, "city": "New York"}
- my_function(**info) #解包字典

*args与**kwargs

- ***args** 将额外的**位置**参数收集为**元组**。
- ****kwargs** 将额外的**关键字**参数收集为**字典**。

- def my_function(*args, **kwargs):
- print("Positional arguments:", args)
- print("Keyword arguments:", kwargs)

- my_function(1, 2, 3, name="Alice", age=25)

*args与**kwargs

```
def describe_pet(animal_type, pet_name, *args, **kwargs):
    print(f"\nI have a {animal_type} named {pet_name}.")
    if args:
        print("Additional info:")
        for info in args:
            print(f"- {info}")
    if kwargs:
        print("Extra details:")
        for key, value in kwargs.items():
            print(f"{key}: {value}")
```

*args与**kwargs

- # 使用混合参数调用函数
- describe_pet(
- "dog",
- "Buddy",
- "Loves to play fetch",
- "Very friendly",
- age=5,
- color="brown"
-)

*args与**kwargs

```
def make_pizza(size, *toppings, crust="regular"):
    print(f"\nMaking a {size}-inch pizza with {crust} crust.")
    if toppings:
        print("Toppings:")
        for topping in toppings:
            print(f"- {topping}")

# Calling the function
make_pizza(12, "pepperoni", "mushrooms", crust="thin")
```

装饰器函数

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Something before the function")

        result = func(*args, **kwargs)

        print("Something after the function")

        return result
    return wrapper
```

创建一个类(class)

```
class student:
 def __init__(self, name, age, **kwargs):
        self.name = name
        self.age = age
        for key, value in kwargs.items():
            setattr(self, key, value)

    def __call__(self):
        print(f"My name is {self.name}.")

def show_age(self):
        print(f"{self.name} is {self.age} years old.")
```

类的用法

- Peter = student(‘peter’, 21)
- Peter
- Peter.__str__()
- Peter()
- Peter.__dict__
- Peter.show_age()

上下文管理器

- with open("file.txt", "r") as file:
- content = file.read()

- with open("example.txt", "w") as file:
- file.write("Hello, World!")

模块化

- 模块是一个包含 Python 定义和语句的文件。包含函数、类、变量以及可执行的代码。
- 每个模块都有自己的命名空间(namespace)，可以避免模块中的变量、函数和类不会与其他模块中的同名实体冲突。
- **可重用性：** 可以在多个项目中重复使用模块，节省开发时间，提高开发效率。
- **可维护性**：将一个大型程序分解为多个模块可以使代码更易于维护。每个模块都可以独立开发、测试和调试。
- **可理解性**：每个模块都有一个明确的目的和功能，更容易地理解整个程序的结构和逻辑。

模块化

- if __name__=='__main__':
- 编写 Python 脚本时，我们希望某些代码只在该脚本作为主程序运行时执行，而不是在被其他模块导入时执行。上述语句就可以用来确定当前模块是否是主程序入口点。
- 在开发过程中，可以在模块内部编写一些测试代码或者示例代码，并将它们放在if __name__=='__main__'条件块内。这样，当你运行该模块作为主程序时，这些测试代码将被执行，而当该模块被其他模块导入时，这些测试代码不会干扰其他模块的正常运行。